Práctica 2: Trabajando con modelos abstractos en Pyomo.

1. Introducción

En esta práctica se trabajará con problemas de optimización utilizando **modelos abstractos**. Esta forma de trabajar permitirá separar el modelo matemático general de los datos concretos de un problema particular (instancia).

La mejor manera de entender el proceso es a través de ejemplos, pero se pueden ver más detalles en el manual online de Pyomo: https://pyomo.readthedocs.io/en/stable/working_abstractmodels/index.html.

2. Trabajando con modelos abstractos en Pyomo

Recordar que para resolver un problema de optimización utilizando el modelo abstracto se necesitan dos ficheros con el siguiente contenido:

- Uno con el modelo matemático genérico: `mi_model.py`
- Otro con los datos concretos del problema a resolver: `mis_datos.dat`

Después se ejecuta en el *prompt* de Anaconda el siguiente comando:

```
pyomo solve --solver='nombre_solver' --summary mi_model.py mis_datos.dat
```

El resultado de la optimización se almacena en el fichero `results.yml` y, al añadir la opción `--summary`, también lo muestra en pantalla.

Si se quiere usar el servidor NEOS, teniendo ya creada la variable de entorno `NEOS_EMAIL` (ver práctica anterior) habría que añadir a la línea de comandos anterior la opción

```
--solver-manager=neos
```

3. Ejemplo Pinturas genérico

En el siguiente ejemplo vamos a generalizar el problema de la fábrica de pinturas que analizamos en la práctica anterior.

Ejemplo 3.1. *Una empresa fabrica pinturas n tipos de pinturas a partir de m materias primas diferentes. Los distintos datos relativos a la producción y venta de estos tipos de pinturas vienen dados por los siguientes parámetros:*

- p_j*: Beneficio por tonelada del tipo de pintura* j, $j = 1, \ldots, n$.
- $disp_i$*: Disponibilidad diaria de materia prima* i, $i = 1, \ldots, m$.
- $prod_{ij}$*: Cantidad necesaria (en ton.) de la materia prima* i *para producir* 1 *tonelada de la pintura* j, $i = 1, \ldots, m$ *y* $j = 1, \ldots, n$.
- P_min_j *y* P_max_j*: Cantidades mínima y máxima, respectivamente, de pintura* j *a producir,* $j = 1, \ldots, n$.

Además, una encuesta de mercado indica que la producción diaria de pintura j *con* $j = 2, \ldots, n$ *no puede ser mayor que* dif_j *tonelada más que la de la pintura* 1.

Al comienzo de cada día, dependiendo de los parámetros anteriores, la empresa desea determinar ***la mezcla de producción óptima*** *de pinturas que maximice su beneficio total diario.*

Solución: La formulación genérica de este problema es la siguiente:

$$
\begin{aligned}
\textbf{maximizar} \quad & z = \sum_{j=1}^{n} p_j x_j \\
\textbf{sujeto a} \quad & \sum_{j=1}^{n} \text{prod}_{ij} x_j \leq \text{disp}_i \qquad i = 1, \ldots, m \\
& x_j - x_1 \leq \text{dif}_j \qquad j = 2, \ldots, n \\
& \text{P_min}_j \leq x_j \leq \text{P_max}_j \qquad j = 1, \ldots, n
\end{aligned}
$$

donde $x_j \in \mathbb{R}$ representa las toneladas a fabricar de pintura j para $j = 1, \ldots, n,$.

El problema abstracto se almacena en el fichero `pinturas_abstract_v1.py` (se puede descargar de la página web de la asignatura) y su contenido será:

```python
import pyomo.environ as pyo

# Crear el modelo
model = pyo.AbstractModel()

# Definir los Parámetros
model.m = pyo.Param(within=pyo.NonNegativeIntegers) #número materias primas diferentes
model.n = pyo.Param(within=pyo.NonNegativeIntegers) #número de tipos de pinturas

model.I = pyo.RangeSet(1, model.m) # Conjunto de Materias primas
model.J = pyo.RangeSet(1, model.n) # Conjunto de Tipos de pintura

model.p = pyo.Param(model.J)  # Precio de venta de cada tipo de pintura
model.prod = pyo.Param(model.I, model.J) # Consumo que hace de la materia prima i la pintura j
model.disp = pyo.Param(model.I) # Disponibilidad de cada materia prima
model.dif = pyo.Param(model.J-[1])  # Diferencia máxima de producción con respecto a la
    pintura 1.

model.P_min = pyo.Param(model.J, within=pyo.Any) # demanda mínima a producir de cada pintura
model.P_max = pyo.Param(model.J, within=pyo.Any) # demanda máxima a producir de cada pintura

# se define la expresión de los pares de (cota_inferir, cota_superior) para las variables
def Pbounds(model, j):
return (model.P_min[j], model.P_max[j])

# Definir las variables y su dominio
model.x = pyo.Var(model.J, bounds=Pbounds, domain=pyo.NonNegativeReals)

# Establecer la función objetivo en dos pasos:
# 1. se define la expresión de la función objetivo
def obj_expression(m):
return pyo.summation(m.p, m.x)
# 2. Se establece la función objetivo
model.OBJ = pyo.Objective(rule=obj_expression,sense=pyo.maximize)

# Establecer el bloque de restricciones sobre las materias primas:
# 1. se define la expresión del bloque de restricciones
def mp_constraint_rule(m, i):
# devuelve la expresión para la restricción (materia prima) i
return sum(m.prod[i,j] * m.x[j] for j in m.J) <= m.disp[i]

# 2. Se crea una restricción para cada materia prima
model.M_P_Constraint = pyo.Constraint(model.I, rule=mp_constraint_rule)

# Establecer el bloque de restricciones sobre la diferencia de producción con la pintura 1:
# 1. se define la expresión del bloque de restricciones
def dp_constraint_rule(m, j):
# devuelve la expresión para la restricción (pintura) j
return m.x[j] - m.x[1] <= m.dif[j]

# 2. Se crea una restricción para cada pintura j DISTINTA de la pintura 1
```

```
model.D_P_Constraint = pyo.Constraint(model.J-[1], rule=dp_constraint_rule)
```

Para resolver un ejemplo concreto de este problema (instancia) se necesitará un fichero con los datos del mismo. La sintaxis que utiliza Pyomo para los ficheros de datos es similar a la utilizada por AMPL[1]. Así, para resolver el problema utilizando los datos del ejemplo de pinturas de la práctica anterior, tendríamos el siguiente fichero de datos `pinturas_abstract_v1_ampl.dat` (se puede descargar de la web de la asignatura):

```
## Ejemplo Pinturas versión 2 - Data en formato AMPL

param n:= 2;
param m:= 2;

param p:=
1 5
2 4;
param P_max:=
1 Infinity
2 2;
param P_min:=
1 0
2 0;
param disp:=
1 24
2 6;

param prod: 1   2:=
1    6   4
2    1   2;

param dif:=
2 1;
```

Resolución del problema utilizando Pyomo:

- **Optimizadores locales:** Se utilizará el optimizador `glpk` y para ello se tiene que ejecutar el siguiente código en el *prompt* de Anaconda:

  `pyomo solve --solver=glpk --summary pinturas_abstract_v1.py pinturas_abstract_v1_ampl.dat`

  El resultado se almacenará en el fichero `result.yml` y se mostrará por pantalla.

- **Servidor NEOS:** Se utilizará, por ejemplo, el optimizador `cplex` y se tiene que ejecutar el siguiente código en el *prompt* de Anaconda[2]:

`pyomo solve --solver-manager=neos --solver=cplex --summary pinturas_abstract_v1.py pinturas_abstract_v1_ampl.dat`

  El resultado se enviará a la dirección de email y se mostrará por pantalla.

□

4. Ejercicios

1. Crear un fichero con el modelo genérico para el problema de Netflix que está en las transparencias del Tema 3.
2. Resolver el problema de Netflix para tres conjuntos de datos creados por vosotros.

[1] Otro lenguaje de modelado: https://ampl.com/
[2] Recordar haber generado primero la variable de entorno `NEOS_EMAIL` con una dirección de correo válida